

Di seguito trovi una **traccia completa di esercitazione d'esame** (stile prova pratica) con **soluzione completa**, pensata per ripassare **Java OOP + Collections + File + JDBC + Spring Boot**.

Puoi usarla come **compito d'esame simulato da 3-4 ore**.



TRACCIA D'ESAME COMPLETA (Java → Spring Boot)

Scenario

Una scuola vuole realizzare un sistema informatico per gestire gli **Studenti** e i loro **Esami**.

Il sistema deve permettere:

- gestione oggetti in memoria (OOP)
 - gestione tramite collezioni
 - salvataggio su file CSV
 - gestione dati su database tramite JDBC (DAO)
 - esposizione API REST tramite Spring Boot
-



PARTE A — JAVA OOP (Obbligatoria)

A1) Classe Studente

Creare una classe `Studente` con:

Attributi

- `id` (int)
- `nome` (String)
- `cognome` (String)
- `classe` (String) es: "5A"
- `media` (double)

Metodi richiesti

- costruttore completo
 - getter e setter
 - `toString()`
 - metodo `boolean promosso()` (true se media $\geq$ 6)
-

A2) Classe Esame

Creare una classe `Esame` con:

Attributi

- `id` (int)
- `studenteId` (int)
- `materia` (String)
- `voto` (double)

Metodi richiesti

- costruttore
- getter e setter
- `toString()`

SOLUZIONE PARTE A

```
public class Studente {
    private int id;
    private String nome;
    private String cognome;
    private String classe;
    private double media;

    public Studente(int id, String nome, String cognome, String classe,
double media) {
        this.id = id;
        this.nome = nome;
        this.cognome = cognome;
        this.classe = classe;
        this.media = media;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNome() { return nome; }
    public void setName(String nome) { this.nome = nome; }

    public String getCognome() { return cognome; }
    public void setCognome(String cognome) { this.cognome = cognome; }

    public String getClasse() { return classe; }
    public void setClasse(String classe) { this.classe = classe; }

    public double getMedia() { return media; }
    public void setMedia(double media) { this.media = media; }

    public boolean promosso() {
        return media >= 6.0;
    }
}
```

```

@Override
public String toString() {
    return "Studente{" +
        "id=" + id +
        ", nome='" + nome + '\'' +
        ", cognome='" + cognome + '\'' +
        ", classe='" + classe + '\'' +
        ", media=" + media +
        '}';
}
}

```

```

public class Esame {
    private int id;
    private int studenteId;
    private String materia;
    private double voto;

    public Esame(int id, int studenteId, String materia, double voto) {
        this.id = id;
        this.studenteId = studenteId;
        this.materia = materia;
        this.voto = voto;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getStudenteId() { return studenteId; }
    public void setStudenteId(int studenteId) { this.studenteId =
studenteId; }

    public String getMateria() { return materia; }
    public void setMateria(String materia) { this.materia = materia; }

    public double getVoto() { return voto; }
    public void setVoto(double voto) { this.voto = voto; }

    @Override
    public String toString() {
        return "Esame{" +
            "id=" + id +
            ", studenteId=" + studenteId +
            ", materia='" + materia + '\'' +
            ", voto=" + voto +
            '}';
    }
}

```

✓ PARTE B — COLLECTIONS + FUNZIONI (Obbligatoria)

B1) Gestione lista studenti

Creare una classe `GestioneStudenti` che contenga:

Struttura dati

- `ArrayList<Studente> studenti`

Metodi richiesti

- `void aggiungiStudente(Studente s)`
- `Studente cercaPerId(int id)`
- `List<Studente> cercaPerClasse(String classe)`
- `void stampaPromossi()`
- `void ordinaPerMediaDecrescente()`

✓ SOLUZIONE PARTE B

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class GestioneStudenti {

    private ArrayList<Studente> studenti = new ArrayList<>();

    public void aggiungiStudente(Studente s) {
        studenti.add(s);
    }

    public Studente cercaPerId(int id) {
        for (Studente s : studenti) {
            if (s.getId() == id) {
                return s;
            }
        }
        return null;
    }

    public List<Studente> cercaPerClasse(String classe) {
        List<Studente> risultato = new ArrayList<>();
        for (Studente s : studenti) {
            if (s.getClasse().equalsIgnoreCase(classe)) {
                risultato.add(s);
            }
        }
        return risultato;
    }
}
```

```

    }
    }
    return risultato;
}

public void stampaPromossi() {
    for (Studiante s : studenti) {
        if (s.promosso()) {
            System.out.println(s);
        }
    }
}

public void ordinaPerMediaDecrescente() {

studenti.sort(Comparator.comparingDouble(Studiante::getMedia).reversed());
}

public void stampaTutti() {
    for (Studiante s : studenti) {
        System.out.println(s);
    }
}

public ArrayList<Studiante> getStudenti() {
    return studenti;
}
}

```

✓ PARTE C — FILE CSV (Obbligatoria)

C1) Scrittura su file CSV

Implementare in `GestioneStudenti` il metodo:

```
void salvaSuFileCSV(String nomeFile)
```

Formato file:

```
id;nome;cognome;classe;media
1;Mario;Rossi;5A;7.5
```

C2) Lettura da file CSV

Implementare:

```
void caricaDaFileCSV(String nomeFile)
```

✓ SOLUZIONE PARTE C

```
import java.io.*;
import java.util.ArrayList;

public void salvaSuFileCSV(String nomeFile) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(nomeFile)))
    {

        bw.write("id;nome;cognome;classe;media");
        bw.newLine();

        for (Studente s : studenti) {
            bw.write(s.getId() + ";" +
                    s.getNome() + ";" +
                    s.getCognome() + ";" +
                    s.getClasse() + ";" +
                    s.getMedia());
            bw.newLine();
        }

    } catch (IOException e) {
        e.printStackTrace();
    }
}

public void caricaDaFileCSV(String nomeFile) {
    studenti = new ArrayList<>();

    try (BufferedReader br = new BufferedReader(new FileReader(nomeFile)))
    {

        String riga = br.readLine(); // intestazione

        while ((riga = br.readLine()) != null) {
            String[] dati = riga.split(";");

            int id = Integer.parseInt(dati[0]);
            String nome = dati[1];
            String cognome = dati[2];
            String classe = dati[3];
            double media = Double.parseDouble(dati[4]);

            Studente s = new Studente(id, nome, cognome, classe, media);
            studenti.add(s);
        }
    }
}
```

```
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

✚ Nota: questa soluzione è volutamente semplice (perfetta per esame).

✓ PARTE D — DATABASE + JDBC + DAO (Obbligatoria)

D1) Creazione Database

Creare una tabella `studenti`:

```
CREATE TABLE studenti (  
    id INT PRIMARY KEY,  
    nome VARCHAR(50),  
    cognome VARCHAR(50),  
    classe VARCHAR(10),  
    media DOUBLE  
);
```

D2) StudenteDAO

Creare una classe `StudenteDAO` con metodi:

- `void insert(Studente s)`
- `List<Studente> findAll()`
- `Studente findById(int id)`
- `void delete(int id)`
- `void updateMedia(int id, double nuovaMedia)`

✓ SOLUZIONE PARTE D

Connessione DB

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
  
public class DBConnection {
```

```

private static final String URL = "jdbc:mysql://localhost:3306/scuola";
private static final String USER = "root";
private static final String PASSWORD = "";

public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(URL, USER, PASSWORD);
}
}

```

DAO completo

```

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class StudenteDAO {

    public void insert(Studente s) {
        String sql = "INSERT INTO studenti(id, nome, cognome, classe, media) VALUES (?, ?, ?, ?, ?)";

        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, s.getId());
            ps.setString(2, s.getNome());
            ps.setString(3, s.getCognome());
            ps.setString(4, s.getClasse());
            ps.setDouble(5, s.getMedia());

            ps.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public List<Studente> findAll() {
        List<Studente> lista = new ArrayList<>();
        String sql = "SELECT * FROM studenti";

        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                Studente s = new Studente(
                    rs.getInt("id"),
                    rs.getString("nome"),

```



```

        rs.getString("cognome"),
        rs.getString("classe"),
        rs.getDouble("media")
    );
    lista.add(s);
}

} catch (SQLException e) {
    e.printStackTrace();
}

return lista;
}

public Studente findById(int id) {
    String sql = "SELECT * FROM studenti WHERE id = ?";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, id);

        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            return new Studente(
                rs.getInt("id"),
                rs.getString("nome"),
                rs.getString("cognome"),
                rs.getString("classe"),
                rs.getDouble("media")
            );
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return null;
}

public void delete(int id) {
    String sql = "DELETE FROM studenti WHERE id = ?";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, id);
        ps.executeUpdate();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

public void updateMedia(int id, double nuovaMedia) {
    String sql = "UPDATE studenti SET media = ? WHERE id = ?";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setDouble(1, nuovaMedia);
        ps.setInt(2, id);

        ps.executeUpdate();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

✓ PARTE E — MAIN DI TEST (Obbligatoria)

E1) Programma di test

Creare un `Main` che:

- crea 3 studenti
- li salva su file CSV
- li ricarica dal file
- inserisce gli studenti nel DB
- stampa tutti gli studenti dal DB
- modifica la media di uno studente

✓ SOLUZIONE PARTE E

```

import java.util.List;

public class Main {

    public static void main(String[] args) {

        GestioneStudenti gs = new GestioneStudenti();

        gs.aggiungiStudente(new Studente(1, "Mario", "Rossi", "5A", 7.5));
        gs.aggiungiStudente(new Studente(2, "Luigi", "Verdi", "5A", 5.0));
        gs.aggiungiStudente(new Studente(3, "Anna", "Bianchi", "4B", 8.2));

        System.out.println("=== STUDENTI IN MEMORIA ===");
    }
}

```

```

        gs.stampaTutti();

        gs.salvaSuFileCSV("studenti.csv");

        GestioneStudenti gs2 = new GestioneStudenti();
        gs2.caricaDaFileCSV("studenti.csv");

        System.out.println("\n=== STUDENTI CARICATI DA FILE ===");
        gs2.stampaTutti();

        StudenteDAO dao = new StudenteDAO();

        for (Studente s : gs2.getStudenti()) {
            dao.insert(s);
        }

        System.out.println("\n=== STUDENTI DAL DATABASE ===");
        List<Studente> listaDB = dao.findAll();
        for (Studente s : listaDB) {
            System.out.println(s);
        }

        System.out.println("\n=== UPDATE MEDIA ID=2 ===");
        dao.updateMedia(2, 6.5);

        System.out.println("\n=== STUDENTE ID=2 ===");
        System.out.println(dao.findById(2));
    }
}

```

PARTE F — SPRING BOOT REST API (Parte avanzata)

F1) Creare API REST

Creare un progetto Spring Boot con dipendenze:

- Spring Web
- Spring Data JPA
- H2 (o MySQL)

Entity `StudenteEntity`

Campi uguali alla classe `Studente`.

F2) Repository

Creare `StudenteRepository` extends `JpaRepository<StudenteEntity, Integer>`

F3) Service

Creare `StudenteService` con metodi:

- `List<StudenteEntity> findAll()`
- `StudenteEntity findById(int id)`
- `StudenteEntity save(StudenteEntity s)`
- `void delete(int id)`

F4) Controller REST

Rotte richieste:

- `GET /api/studenti`
- `GET /api/studenti/{id}`
- `POST /api/studenti`
- `DELETE /api/studenti/{id}`

✓ SOLUZIONE PARTE F (Spring Boot)

Entity

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class StudenteEntity {

    @Id
    private int id;

    private String nome;
    private String cognome;
    private String classe;
    private double media;

    public StudenteEntity() {}

    public StudenteEntity(int id, String nome, String cognome, String
classe, double media) {
        this.id = id;
        this.nome = nome;
        this.cognome = cognome;
        this.classe = classe;
        this.media = media;
    }
}
```

```
public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getNome() { return nome; }
public void setNome(String nome) { this.nome = nome; }

public String getCognome() { return cognome; }
public void setCognome(String cognome) { this.cognome = cognome; }

public String getClasse() { return classe; }
public void setClasse(String classe) { this.classe = classe; }

public double getMedia() { return media; }
public void setMedia(double media) { this.media = media; }
}
```

Repository

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudenteRepository extends JpaRepository<StudenteEntity,
Integer> {
}
```

Service

```
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class StudenteService {

    private final StudenteRepository repo;

    public StudenteService(StudenteRepository repo) {
        this.repo = repo;
    }

    public List<StudenteEntity> findAll() {
        return repo.findAll();
    }

    public StudenteEntity findById(int id) {
        return repo.findById(id).orElse(null);
    }

    public StudenteEntity save(StudenteEntity s) {
```

```
        return repo.save(s);
    }

    public void delete(int id) {
        repo.deleteById(id);
    }
}
```

Controller REST

```
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/studenti")
public class StudenteController {

    private final StudenteService service;

    public StudenteController(StudenteService service) {
        this.service = service;
    }

    @GetMapping
    public List<StudenteEntity> getAll() {
        return service.findAll();
    }

    @GetMapping("/{id}")
    public StudenteEntity getById(@PathVariable int id) {
        return service.findById(id);
    }

    @PostMapping
    public StudenteEntity create(@RequestBody StudenteEntity s) {
        return service.save(s);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable int id) {
        service.delete(id);
    }
}
```

▣ PARTE G — DOMANDE TEORICHE (da fare a voce)

Domande tipiche (con risposta attesa breve)

1. Differenza tra `ArrayList` e `LinkedList`
 2. Cos'è il polimorfismo
 3. `==` vs `equals()`
 4. Perché usare `PreparedStatement`
 5. Cos'è un DAO
 6. Cos'è una REST API
 7. Differenza tra `@Controller` e `@RestController`
 8. Cos'è JPA
 9. Differenza tra `Entity` e `DTO`
 10. Perché Spring usa Dependency Injection
-



VALUTAZIONE (griglia esame)

- OOP + correttezza classi: **20%**
- Collections + funzioni richieste: **20%**
- File CSV: **15%**
- JDBC + DAO: **25%**
- Spring Boot REST: **20%**